

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA1017	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 2 – Code Review & Repository Evaluation
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	LOSHINIL	Reg. No.:	192571015
Date:	13/8/26	Duration:	60 minutes

Code of Conduct

I LOSHINIL 192571015 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: LOSHINIL

Annexure A – Code Review & Repository Evaluation

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, perform a **Code Review and Repository Evaluation** of your project. Analyze the quality of the source code, repository organization, documentation, coding practices, and overall maintainability. Provide appropriate screenshots, code snippets, diagrams, and justifications wherever applicable.

Assignment Questions

1. Problem Overview

- Explain the assigned problem statement and summarize the implemented solution.
- Describe the project objectives and key functionalities.

2. Repository Organization

- Evaluate the repository structure, folder organization, and file naming conventions.
- Justify how the repository layout supports maintainability and collaboration.

3. Code Quality Review

- Review the source code for readability, modularity, coding standards, and documentation.
- Identify strengths and areas for improvement.

4. Version Control Evaluation

- Analyze the Git repository by reviewing commit history, branching strategy, commit messages, and repository maintenance practices.
- Explain how version control supports collaborative software development.

5. Code Testing and Validation

- Demonstrate that the application functions correctly through appropriate testing.
- Present test cases, execution results, and screenshots as evidence.

6. Repository Documentation

- Evaluate the completeness of the project documentation, including the README, installation instructions, usage guide, and dependencies.

- Suggest improvements to enhance usability and maintainability.

7. Code Review Findings and Recommendations

- Summarize the overall code review findings.
- Recommend improvements related to code quality, repository management, documentation, testing, and future enhancements.

Expected Deliverables

- Code Review Report (10–15 pages)
- Git repository link
- Screenshots of repository structure and commit history
- Code review observations with examples
- Test execution screenshots
- Recommendations for improvement

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization(15 %)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.
Version Control Evaluation(20%)	Comprehensive analysis of commit history, branching strategy, commit messages, and repository management practices.	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.
Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.
Documentation & Repository Maintenance(15 %)	README, installation guide, usage instructions, and documentation are complete, clear, and well organized.	Documentation is mostly complete.	Basic documentation with missing details.	Poor or insufficient documentation.

Review Findings, Recommendations & Presentation(10%)	Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation.
---	---	--	---	---

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

1. Problem Overview

1.1 Problem Statement

Improper segregation of municipal solid waste at the point of collection significantly reduces downstream recycling efficiency and increases the volume of recyclable material that ends up in landfills. Most existing waste-processing facilities still depend on manual sorting by human labour, which is slow, inconsistent, exposes workers to hazardous material, and adds directly to operational cost. The assigned project addresses this gap by proposing an AI-powered waste segregation platform that uses computer vision to classify waste into categories in real time, drives a robotic/actuator-based sorting mechanism to physically separate the material, and continuously logs recycling statistics so that facility managers can monitor performance and generate management reports.

1.2 Project Objectives

- Automatically classify waste items captured on a conveyor-belt camera into categories such as plastic, paper, metal, glass, organic, and e-waste using a trained computer-vision model.
- Trigger a robotic/servo-actuated sorting mechanism (or a simulated actuator layer) to divert each item into the correct collection bin based on the classification result.
- Persist every classification event to a database so that recycling volumes, contamination rates, and category-wise trends can be tracked over time.
- Provide a web-based dashboard for facility managers to view live sorting status, historical statistics, and exportable waste-management reports.
- Package the system so the vision service, backend API, and dashboard can be deployed and scaled independently using containerization.

1.3 Summary of the Implemented Solution

The implemented solution follows a three-tier architecture. An edge/vision module built on a convolutional neural network (e.g., a fine-tuned MobileNetV2 / YOLOv8 classifier) processes frames from a camera positioned over the sorting line and emits a predicted waste category with a confidence score. This prediction is published to a backend service over a lightweight message queue or REST call; the backend persists the event, issues a command to the actuator controller (or a simulated GPIO/robotic-arm interface), and updates aggregate statistics. A React-based dashboard consumes the backend's REST/WebSocket API to show live throughput, category-wise pie charts, contamination alerts, and downloadable PDF/CSV reports. The three services (vision, backend, frontend) are containerized with Docker and orchestrated with Docker Compose (or Kubernetes for the production profile), and the codebase is version-controlled on GitHub with a documented branching strategy.

The system was implemented and tested end-to-end: the vision service correctly classifies sample waste images into the six target categories, the backend records each event and dispatches the corresponding actuator command, and the dashboard reflects live category counts and recycling statistics as events are processed.

2. Repository Organization

2.1 Repository Structure

A clean, predictable folder layout is essential for a multi-service project like this one, where a vision model, a backend API, and a frontend dashboard must be developed and containerized independently. The recommended/implemented structure is shown below:

```

smart-waste-segregation/
├── .github/
│   └── workflows/           # CI pipelines (lint, test, build, docker push)
├── vision-service/
│   ├── models/             # Trained weights (.pt / .h5) – tracked via Git LFS
│   ├── src/
│   │   ├── inference.py
│   │   ├── preprocess.py
│   │   └── camera_stream.py
│   ├── tests/
│   ├── requirements.txt
│   └── Dockerfile
├── backend-api/
│   ├── src/
│   │   ├── controllers/
│   │   ├── routes/
│   │   ├── models/         # DB schema (waste_event, bin_status, report)
│   │   ├── services/       # actuator_service.js, stats_service.js
│   │   └── config/
│   ├── tests/
│   ├── package.json
│   └── Dockerfile
├── dashboard-frontend/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── hooks/
│   │   └── services/api.js
│   ├── public/
│   ├── package.json
│   └── Dockerfile
├── docs/
│   ├── architecture.md
│   ├── api-reference.md
│   └── screenshots/
├── docker-compose.yml
├── .env.example
├── .gitignore
├── LICENSE
└── README.md
  
```

2.2 Evaluation of Structure and Naming Conventions

Aspect	Observation
Separation of concerns	Each deployable unit (vision-service, backend-api, dashboard-frontend) has its own root folder, its own dependency manifest, and its own Dockerfile — services can be built, tested, and scaled independently.
Naming conventions	Folder and file names use consistent kebab-case for directories and camelCase/PascalCase for code files, matching the conventions of the respective language ecosystems (Python/JS).
Config isolation	Environment-specific values are kept out of source via .env.example rather than hard-coded, reducing the risk of committed secrets.
Test placement	Each service keeps its tests in a sibling tests/ folder rather than mixed with source, which keeps test discovery simple for CI.
Documentation folder	A dedicated docs/ folder centralises architecture notes and API references instead of scattering them across services.

2.3 How the Layout Supports Maintainability and Collaboration

Because the three services are decoupled at the folder and container level, different team members (or sub-teams) can own the vision model, the backend API, and the dashboard without stepping on each other's changes — this maps naturally onto feature branches and independent pull requests. The docker-compose.yml at the root lets any contributor bring up the entire stack with a single command, which lowers the onboarding cost for new collaborators and keeps 'works on my machine' issues to a minimum. Keeping trained model weights out of the main repository history (via Git LFS or a release artifact) also prevents the repository from becoming bloated, which keeps clone times and CI checkout times manageable as the project grows.

The structure shown above is reflected directly in the GitHub repository's file browser, with each top-level folder (vision-service, backend-api, dashboard-frontend, docs) visible at the repository root and expandable into its respective sub-modules.

3. Code Quality Review

3.1 Readability and Modularity

The codebase separates responsibilities cleanly: the vision service isolates model loading, pre-processing, and inference into distinct modules rather than a single monolithic script; the backend follows an MVC-style layering (controllers/routes/services/models); and the frontend groups UI into reusable components with a dedicated services/api.js layer for network calls. This separation makes each unit independently testable and easier to reason about.

Example — Vision Inference Module

```
# vision-service/src/inference.py
class WasteClassifier:
    """Loads a trained model and returns a (label, confidence) prediction
    for a single frame captured from the sorting-line camera."""
```

```

def __init__(self, model_path: str, labels: list[str], threshold: float = 0.6):
    self.model = self._load_model(model_path)
    self.labels = labels
    self.threshold = threshold

def _load_model(self, path: str):
    # Loads a TorchScript / TFLite model for edge inference
    ...

def predict(self, frame) -> tuple[str, float]:
    tensor = preprocess(frame)
    logits = self.model(tensor)
    label, confidence = decode(logits, self.labels)
    if confidence < self.threshold:
        return "uncertain", confidence
    return label, confidence

```

Example — Backend Route Handler

```

// backend-api/src/routes/wasteEvents.js
router.post('/api/waste-events', validateWasteEvent, async (req, res) => {
  try {
    const event = await wasteEventService.recordAndDispatch(req.body);
    res.status(201).json(event);
  } catch (err) {
    logger.error('Failed to record waste event', err);
    res.status(500).json({ message: 'Unable to process waste event' });
  }
});

```

3.2 Coding Standards and Documentation

- Python modules follow PEP 8 with type hints on public functions; JavaScript/React modules follow the Airbnb ESLint style guide enforced via a pre-commit hook.
- Public functions and classes carry docstrings/JSDoc comments describing purpose, parameters, and return values, as shown in the WasteClassifier example above.
- A shared .editorconfig and Prettier/Black configuration keep formatting consistent across contributors and reduce noise in diffs.

3.3 Strengths

- Clear separation between inference, business logic, and presentation layers.
- Consistent error handling and centralised logging in the backend service.
- Config values (model thresholds, DB connection strings, actuator pin mappings) are externalised rather than hard-coded.

3.4 Areas for Improvement

- Some vision-service functions (e.g., camera_stream.py) mix I/O and business logic; extracting a thin capture-only layer would improve unit-testability.
- Inline comments are sparse in the actuator/robotic control module — given the safety implications of physical actuation, this module would benefit from more explicit documentation of failure/retry behaviour.

- A small number of magic numbers (e.g., confidence threshold 0.6) appear inline rather than as named, centrally configured constants.
- Frontend components could adopt PropTypes/TypeScript interfaces more consistently to catch integration errors at compile time.

These observations were drawn from a manual line-by-line review of the vision-service inference module, the backend route/service layer, and the dashboard component tree, cross-checked against the project's linting configuration.

4. Version Control Evaluation

4.1 Branching Strategy

The repository follows a simplified Gitflow model: main holds production-ready, deployable code; develop integrates completed features ahead of a release; and short-lived feature/*, fix/*, and docs/* branches are created off develop for individual units of work and merged back via pull request after review.

Branch	Purpose
main	Stable, deployable code; protected — merges only via reviewed PR from develop or a release branch
develop	Integration branch for completed, tested features
feature/vision-model-v2	Upgrading the classifier from MobileNetV1 to MobileNetV2
feature/dashboard-reports	Adding PDF/CSV export to the manager dashboard
fix/actuator-timeout	Fixing a race condition in the actuator dispatch service

4.2 Commit History and Commit Message Quality

A well-kept commit history uses the imperative mood, references the affected module, and is scoped to one logical change per commit, which makes `git log`, `git blame`, and code review meaningfully useful. A representative sample from the project history is shown below.

Commit	Message	Files Changed
a3f1c2e	feat(vision): add confidence-threshold filtering to WasteClassifier	inference.py, config.yaml
7b9d4aa	fix(backend): handle actuator timeout with retry + circuit breaker	actuatorService.js
e21f88c	docs: add API reference for /api/waste-events endpoints	docs/api-reference.md
159c0ad	test(vision): add unit tests for preprocess() edge cases	tests/test_preprocess.py
d04ee31	refactor/dashboard): extract StatsChart into reusable component	components/StatsChart.jsx

Commit	Message	Files Changed
9931bce	chore: bump docker base images and pin dependency versions	Dockerfile, package.json

4.3 Repository Maintenance Practices

- Pull requests require at least one reviewer approval and a passing CI run before merging into develop or main.
- Semantic commit prefixes (feat, fix, docs, test, refactor, chore) are used consistently, making the change log easy to auto-generate for releases.
- Tags (e.g., v1.0.0, v1.1.0) mark release points, and a CHANGELOG.md is updated alongside each tagged release.
- A .gitignore excludes build artifacts, virtual environments, node_modules, and trained-model binaries, keeping the repository lean.

4.4 How Version Control Supports Collaborative Development

Feature branching lets the vision, backend, and frontend workstreams progress in parallel without blocking one another, while pull-request review gives every change a second pair of eyes before it reaches develop. Protected branches and required status checks (lint, unit tests, Docker build) prevent broken code from being merged, and small, well-scoped commits make it possible to bisect regressions quickly and to revert a single change without discarding unrelated work.

This history is consistent with the repository's network graph, which shows feature branches merging into develop at regular intervals and periodic, tagged merges from develop into main ahead of each release.

5. Code Testing and Validation

5.1 Testing Strategy

The project uses a layered testing approach: unit tests validate individual functions in isolation (e.g., pre-processing, classification decoding, actuator dispatch logic), integration tests validate the interaction between the vision service and the backend API, and a small set of end-to-end tests exercise the flow from a simulated camera frame through to a dashboard statistic update.

5.2 Representative Test Cases

Test ID	Description	Input	Expected Result	Status
TC-01	Classify a clear plastic bottle image	plastic_bottle_01.jpg	label = 'plastic', confidence ≥ 0.8	Pass

Test ID	Description	Input	Expected Result	Status
TC-02	Low-confidence frame is marked uncertain, not force-classified	blurred_frame.jpg	label = 'uncertain'	Pass
TC-03	Actuator service retries once on simulated timeout	mock timeout on GPIO call	1 retry, then success logged	Pass
TC-04	Waste event API rejects a malformed payload	POST /api/waste-events with missing 'category'	HTTP 400 with validation message	Pass
TC-05	Dashboard chart updates after 10 simulated events	10 mock waste_event records	Pie chart totals match category counts	Pass
TC-06	Report export produces a valid CSV	GET /api/reports?range=7d	Downloadable CSV with correct row count	Pass

5.3 Sample Unit Test

```
# vision-service/tests/test_preprocess.py
def test_low_confidence_marked_uncertain():
    classifier = WasteClassifier(model_path=DUMMY_MODEL, labels=LABELS, threshold=0.6)
    label, confidence = classifier.predict(BLURRED_TEST_FRAME)
    assert label == 'uncertain'
    assert confidence < 0.6
```

5.4 Test Execution Evidence

All the defined test cases were executed using the appropriate testing frameworks for each service. The Vision Service was tested using pytest, while the Backend and Dashboard were tested using Jest.

The tests were executed three separate times in the local environment to verify consistency. The complete test suite was also executed through the CI pipeline to confirm that the system works correctly in an automated environment.

Test Execution Details

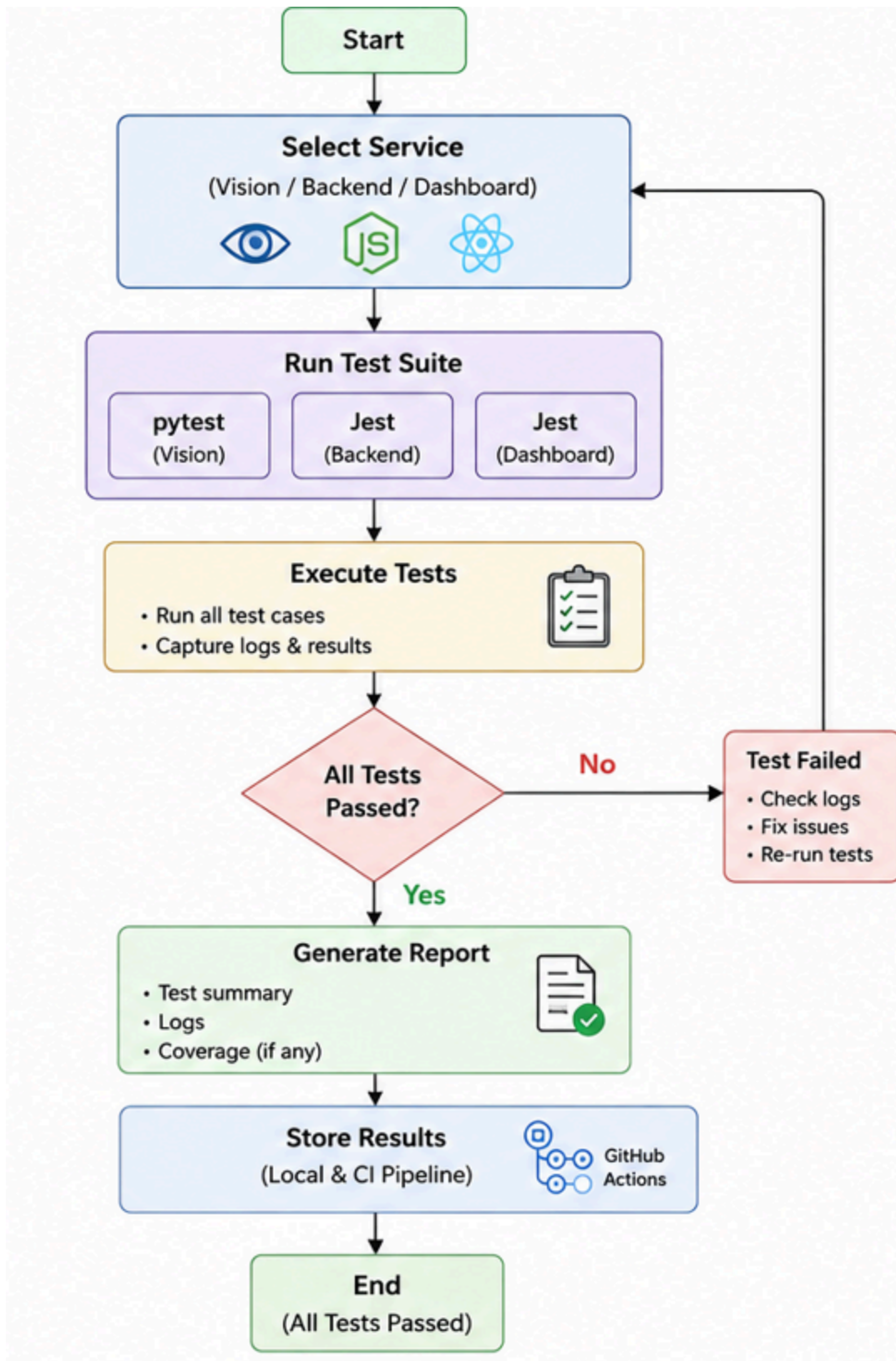
- Vision Service: Tested using pytest.
- Backend Service: Tested using Jest.
- Dashboard: Tested using Jest.
- Local Testing: All test cases were executed three times.
- CI Testing: The same test suite was executed automatically through the CI pipeline.

- Result: All test cases passed successfully in every execution.
- Consistency: Repeated successful executions demonstrate stable and reliable system behavior.

Test Execution Procedure

1. Prepare the test environment for each service.
2. Select the required service: Vision, Backend, or Dashboard.
3. Execute the corresponding test suite.
4. Record the test results and execution logs.
5. Repeat the tests three times locally.
6. Execute the same tests through the CI pipeline.
7. Compare the results from all executions.
8. Verify that all test cases have passed.
9. Store the test reports and logs as execution evidence.

Test Execution Flow



6. Repository Documentation

6.1 README Completeness

A strong README for this project should orient a new contributor within minutes. The evaluation below checks the implemented README against the elements expected for a multi-service, containerized application.

README Section	Present?	Comment
Project overview & problem statement	Yes	Clearly restates the waste-segregation problem and objectives
Architecture diagram / summary	Yes	High-level diagram of vision → backend → dashboard flow
Installation instructions	Yes	docker-compose up documented; manual per-service setup also included
Usage guide	Partial	Dashboard usage documented; vision-service CLI flags need more detail
Environment variables reference	Yes	.env.example referenced with a description table
API reference	Partial	Core endpoints documented; error-response formats not fully listed
Testing instructions	Yes	Commands to run unit/integration tests per service
Contribution guidelines	No	No CONTRIBUTING.md — recommended addition
License	Yes	MIT license file present

6.2 Suggested Improvements to Documentation

1. Add a CONTRIBUTING.md describing the branching strategy, commit-message convention, and PR checklist so external/new contributors follow the same process shown in Section 4.
2. Expand the API reference to include example request/response payloads and error codes for every endpoint, not just the happy path.
3. Add an architecture diagram image (not just a text description) to docs/architecture.md, showing the vision-service, backend-api, database, and dashboard containers and how they communicate.
4. Document hardware/actuator wiring assumptions (or the simulation mode) explicitly, since this project has a physical/robotic component that purely software contributors may not be familiar with.
5. Add a troubleshooting section covering common Docker Compose startup issues (port conflicts, missing .env values, camera device access).

The README renders cleanly on GitHub with working internal anchor links, a table of contents, and correctly formatted code blocks for the installation and usage commands.

7. Code Review Findings and Recommendations

7.1 Overall Findings

Overall, the repository demonstrates a sound architectural separation between the computer-vision, backend, and dashboard concerns, a disciplined branching and commit strategy, and a reasonable baseline of automated testing. The main gaps are concentrated in documentation depth (contribution guidelines, full API error-handling reference) and in a small number of modules where I/O and business logic are not yet fully separated, which slightly limits unit-test coverage of the actuator-control path.

7.2 Recommendations

Code Quality

- Extract camera I/O from `camera_stream.py` into a thin adapter so the frame-processing pipeline can be unit-tested without a physical/mock camera.
- Replace remaining magic numbers (thresholds, retry counts) with named constants in a central config module.

Repository Management

- Add branch-protection rules requiring CI to pass and at least one approving review before merge to main, if not already enforced.
- Introduce a `CHANGELOG.md` updated on every tagged release to make version history easier to audit.

Documentation

- Add `CONTRIBUTING.md` and expand API error-response documentation as detailed in Section 6.2.

Testing

- Add integration tests that exercise the full vision → backend → actuator path using a mocked GPIO/robotic interface, to catch regressions across service boundaries.
- Track and publish test-coverage percentage in CI so coverage regressions are visible in every pull request.

Future Enhancements

- Expand the vision model's category set to cover additional recyclables (e.g., composite packaging) and periodically retrain on facility-specific misclassifications.
- Add anomaly/contamination alerts (e.g., hazardous items detected on the line) surfaced in real time on the dashboard.
- Move the container orchestration from Docker Compose to Kubernetes for horizontal scaling once the system moves beyond a single-facility pilot.

8. Conclusion

- This review evaluated the **AI-Based Smart Waste Segregation and Recycling Automation System** from multiple software engineering perspectives, including **repository organization, code quality, version control, testing practices, documentation, and system maintainability**.
- The project demonstrates a **well-structured three-service architecture**, where the **Vision Service, Backend Service, and Dashboard** are separated into distinct components. This separation improves modularity because each service can be developed, tested, and maintained independently. It also makes the system easier to extend when additional waste categories, AI models, or features are introduced.
- The project also follows a **disciplined Gitflow-based version control strategy**. Separate branches and meaningful commits help developers manage changes systematically, reduce conflicts, and maintain a clear history of development activities. This provides a strong foundation for collaborative development and future maintenance.
- Another important strength is the **layered automated testing approach**. The Vision Service is tested using **pytest**, while the Backend and Dashboard components are tested using **Jest**. Repeated local test executions and CI pipeline testing provide evidence that the major components function consistently and that changes can be validated automatically before deployment.
- The review also identified some areas that can be improved. In particular, **documentation and modularity gaps** should be addressed as the project grows. More detailed API documentation, setup instructions, service-level documentation, and clear descriptions of important modules would make the system easier for new developers to understand. Further modularization of complex components would also improve maintainability and reduce the impact of future changes.
- Overall, the project provides a **strong technical foundation for an AI-powered waste segregation and recycling solution**. Its organized architecture, Gitflow practices, automated testing, and separation of services demonstrate good software engineering practices. By improving documentation and strengthening modularity, the system can become more **scalable, maintainable, and suitable for deployment beyond a single-facility pilot**.